Sir Syed University of Engineering & Technology
Department of Computer Science & Information Technology

CS-467- Software Project Management

Lecture No. 8

AGILE PROJECT MANAGEMENT

Prepared by: Dr. Engr. M. Nadeem

List of Topic to be Discuss in this lecture

- What is Agility?
- Three drivers of Agility
- Origin of Agile Software Development
- Adaptive vs Predictive Approach
- The Agile Manifesto – 4 Core Values
- Agile Manifesto - Twelve Principles
- Agile Methods
- Scrum
- Kaban

What is Agility?

- Agility is the capability to efficiently and effectively adapt to an ever changing environment.
- Agility can be defined at different levels: personal, departmental or organizational.
- Three drivers which will cause a system to become more agile:
  1. Flow
  2. Learning
  3. Collaboration

Three drivers of Agility

- **Flow** refers to the way work is processed by the system. If the processing of work occurs at a steady and sustainable rate, the system has a high level of flow.

- **Learning** refers to mechanisms in the system which allow the system to learn from past experiences, mistakes or other people, but also to uncover unknown unknowns (knowledge discovery).

- **Collaboration** refers to various ways how people in the system can work together to achieve a single goal. Different forms of collaboration can be distinguished

Origin of Agile Software Development

- Traditionally, software development methodologies followed a waterfall approach which resembles traditional project management methods in nature, with a strong focus on processes, predictability and risk control, assuming a stable environment.

- In response to such heavyweight software development methods, various lightweight methods were introduced in the 1990s, such as Scrum, Extreme Programming and Rapid Application Development.

- While such lightweight methods were introduced before the Agile Manifesto, the latter is often considered the start of the Agile movement.

- The Agile Manifesto was written in 2001 by 17 software developers who came together to discuss the lightweight development methods. Together they published the Manifesto for Agile Software Development.

Adaptive vs Predictive Approach

- In a **predictive** approach the focus is on analyzing and planning the future in detail, preparing for potential risks. The idea is that if one can predict the future, one can plan for it.

- In an **adaptive** approach it is assumed that the environment changes fast, often and in unpredictable ways. The focus of adaptive approaches is not to plan everything ahead of time but to leave flexibility to respond to a changing environment. Flexibility to address changes and the ability to detect these changes are key elements in an adaptive approach.

- In a waterfall approach, planning, execution, testing and delivery are clearly separated in sequential steps. In an integrative (also called agile) approach, these steps are integrated in one phase, which is typically iterated multiple times. These iterations are also incremental by nature, which means that each iteration builds upon the output of the previous iteration enhancing it further towards the end deliverable.

Adaptive vs Predictive Approach

The following figures are an excellent example of the differences between traditional (or phased) software development vs. the Agile approach of iterative development.

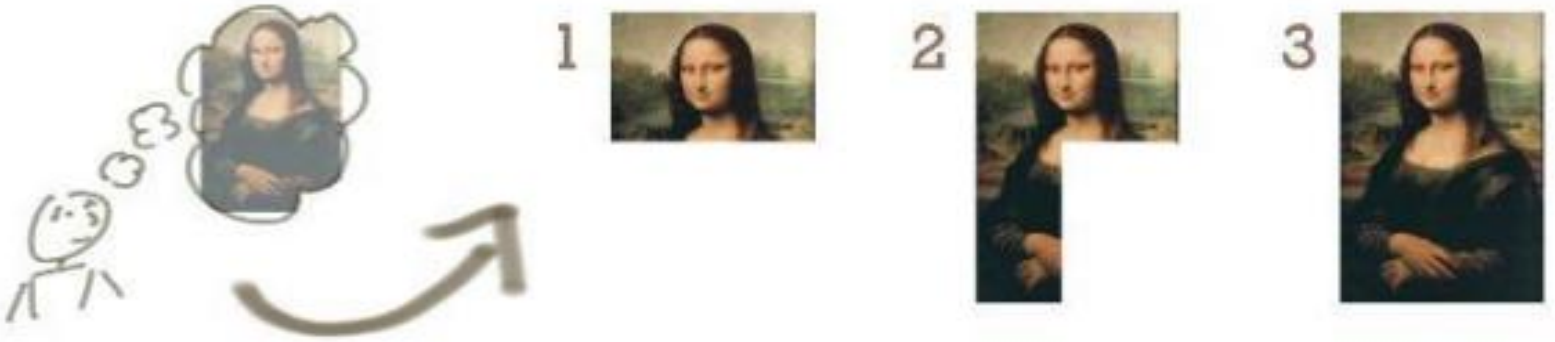


FIGURE 1: THE TRADITIONAL APPROACH (PHASED DELIVERY OF KNOWN OUTPUTS)

FIGURE 2: THE AGILE APPROACH (ITERATIVE DELIVERY TO MEET CHANGING EXPECTATIONS)

The Agile Manifesto – 4 Core Values

The agile manifesto identifies four core values:

- Individuals and interactions over processes and tools. While tools and processes are important, it is even more important to have competent people working together effectively.

- Working software over comprehensive documentation. While good documentation is useful, the main point of development is to create software not documentation.

- Customer collaboration over contract negotiation. While a contract is important, it is no substitute for working closely with customers to discover what they need.

- Responding to change over following plan. While a project plan is important, it should not hinder to accommodate to changes

Agile Manifesto - Twelve Principles

The agile manifesto also identified twelve principles:

1. Customer satisfaction by early and continuous delivery of valuable software.
2. Welcome changing requirements, even in late development.
3. Deliver working software frequently (weeks rather than months).
4. Close, daily cooperation between business people and developers.
5. Projects are built around motivated individuals, who should be trusted.
6. Face-to-face conversation is the best form of communication (co-location).
7. Working software is the primary measure of progress.
8. Sustainable development, able to maintain a constant pace.
9. Continuous attention to technical excellence and good design.
10. Simplicity - the art of maximizing the amount of work not done - is essential.
11. Best architectures, requirements, and designs emerge from self-organizing teams.
12. Regularly, the team reflects on how to become more effective, and adjusts accordingly.

Agile Methods

The term Agile actually refers to a concept, not a specific methodology. There are many, and sometimes conflicting, methods that can be used under the Agile umbrella. These include;

1. **Agile Unified Process**
2. **Behaviour Driven Development (BDD),**
3. **Crystal Clear**
4. **Dynamic Systems Development Method (DSDM)**
5. **Extreme Programming (XP)**
6. **Feature Driven Development (FDD),**
7. **Kanban**
8. **Lean Development,**
9. **Rapid Application Development (RAD)**
10. **IBM - Rational Unified Process (RUP)**
11. **Scrum**
12. **Test Driven Development (TDD),**

Two Prominent Agile Project Management Methodologies

Agile has two prominent methodologies:

1. Scrum
2. Kanban

- In Scrum, the work plans are split into shorter sprints.
- For larger projects that have the potential to be tackled in interactive series, Scrum is preferable.
- In Kanban, there is a continuous release of work times in little time frames.
- Kanban works better for projects with medium to long-term periods.

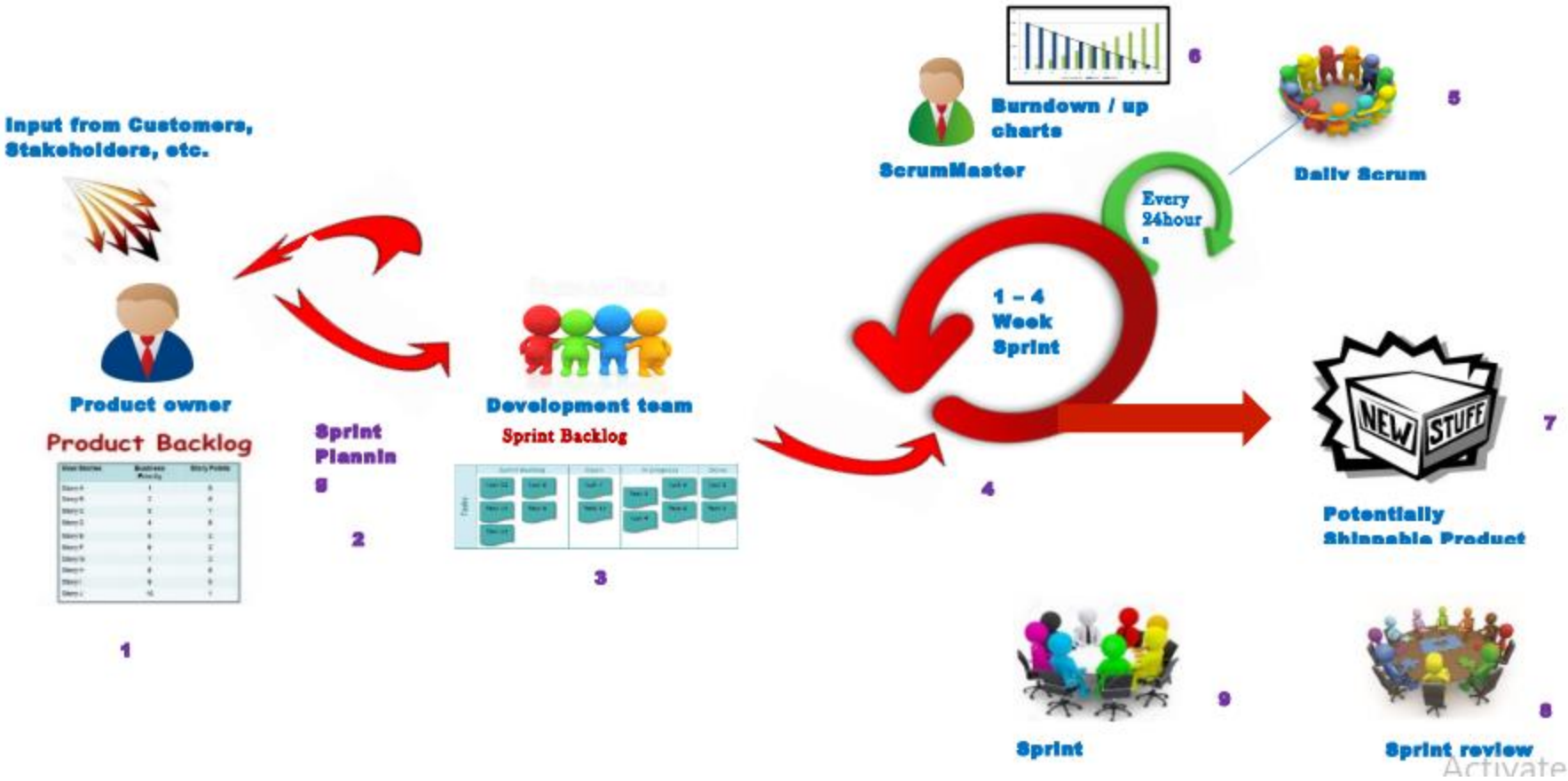
SCRUM OVERVIEW

Scrum is described as a ‘framework within which you can employ various processes and techniques’, rather than a process, or a technique, for building products. The Scrum framework is primarily team based, and defines associated roles, events, artefacts and rules. The three primary roles within the Scrum framework are:

1. The product owner who represents the stakeholders,
2. The scrum master who manages the team and the Scrum process
3. The team about 7 people, who develop the software.

Each project is delivered in a highly flexible and iterative manner where at the end of every sprint of work there is a tangible deliverable to the business.

Typical Flow Diagram of Scrum Framework



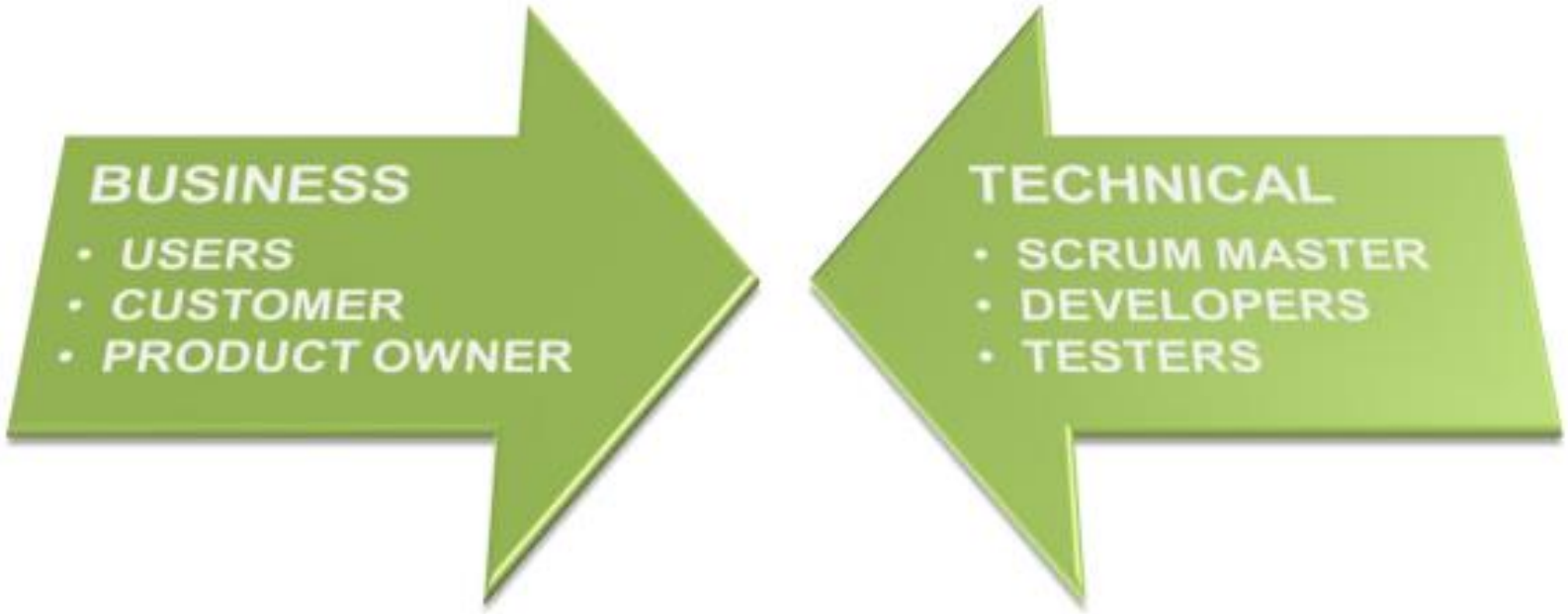
Scrum : Key Scrum Concepts

Concept	Description
Project	Discreet set of end user requirements that have been grouped, prioritised and funded.
Requirement	The end user statement that outlines their information need.
Sprint	A sprint is a 1 to 4 week time-boxed event focused on the delivery of a subset of User Stories taken from the Project Backlog.
Product Backlog	The Project Backlog is the current list of User Stories for the Project. User Stories can be added, modified or removed from the Backlog during the Project.
Sprint Backlog	Subset of User Stories from the Project Backlog that are planned to be delivered as part of a Sprint.
User Stories	The User Story is a one or two line description of the business need, usually described in terms of features.
Tasks	Tasks are the activities performed to deliver a User Story.
Technical Debt	This refers to items that were either: • missing from the Planning meeting; or • deferred in favor of early delivery.

Primary Scrum Roles

Role	Primary Responsibility	Typical	Does Not
Users *Interested Role*	• Use the software • Identify issues & • Provide feedback	• There are no typical users.	• Set Scope • Test Work
Customers *Interested Role*	• Define, start and end the project	• Internal managers • External Clients	• Direct Work
Product Owner *Interested Role*	• Manage the product backlog • Set the scope • Approve Releases	• Project Manager • Product manager • Customer	• Manage the Team
Scrum Master *Committed Role*	• Manage the Agile process • Report on progress	• Project manager • Team Leader • Team member	• Prioritise features
Developers *Committed Role*	• Develop features • Resolve issues	*cross functional* • Developer • Designers • Writers • Administrators	• Prioritise features
Testers *Committed Role*	• Test • Approve or reject features for release	• Existing developers • Dedicated testers	• Test their own code

Business Vs Technical Scrum Roles



Example

Product Backlog:

Let's say below are user stories with story points. Let's say it has been decided to complete all these user stories in 4 sprints, and the team has assumed a velocity of 10 story points per sprint. Let's say it's a 1 week sprint.

Product Backlog		
S.#	**User Stories**	**Estimation (User Story points)**
1	User Stories - I	4
2	User Stories - II	2
3	User Stories - III	4
4	User Stories - IV	2
5	User Stories - V	4
6	User Stories - VI	6
7	User Stories - VII	8
8	User Stories - VIII	6
9	User Stories - IX	4
10	User Stories - X	8

Example

Sprint backlog:

As part of sprint planning, the goal for sprint 1 is to complete first 3 user stories and the same has been moved to the Sprint backlog as below.

Day 1				
Sprint Backlog - Sprint 1				
S.#	User Stories	Estimation (User Story points)	In Progress	Done
1	User Stories - I	4	Yes	
2	User Stories - II	2	Yes	
3	User Stories - III	4		

Day 2				
Sprint Backlog - Sprint 1				
S.#	User Stories	Estimation (User Story points)	In Progress	Done
1	User Stories - I	4	Yes	
2	User Stories - II	2		Yes
3	User Stories - III	4		

Example

Day 3				
Sprint Backlog - Sprint 1				
S.#	User Stories	Estimation (User Story points)	In Progress	Done
1	User Stories - I	4		Yes
2	User Stories - II	2		Yes
3	User Stories - III	4	Yes	

Day 4				
Sprint Backlog - Sprint 1				
S.#	User Stories	Estimation (User Story points)	In Progress	Done
1	User Stories - I	4		Yes
2	User Stories - II	2		Yes
3	User Stories - III	4	Yes	

Day 5				
Sprint Backlog - Sprint 1				
S.#	User Stories	Estimation (User Story points)	In Progress	Done
1	User Stories - I	4		Yes
2	User Stories - II	2		Yes
3	User Stories - III	4		Yes

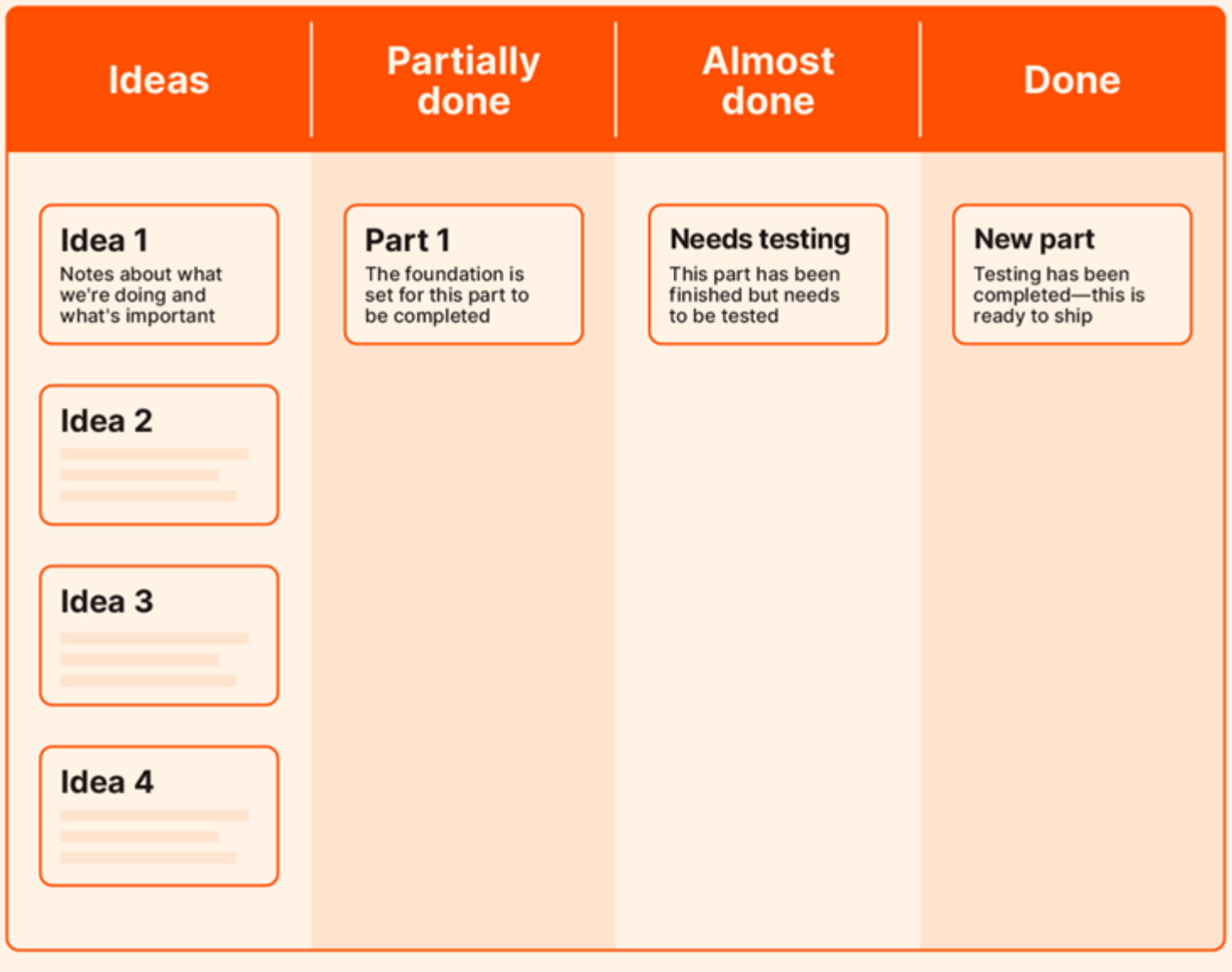
Kanban Overview

Kanban isn't a separate methodology from Agile: it's an approach that can be used in tandem with Agile and other flexible project management styles.

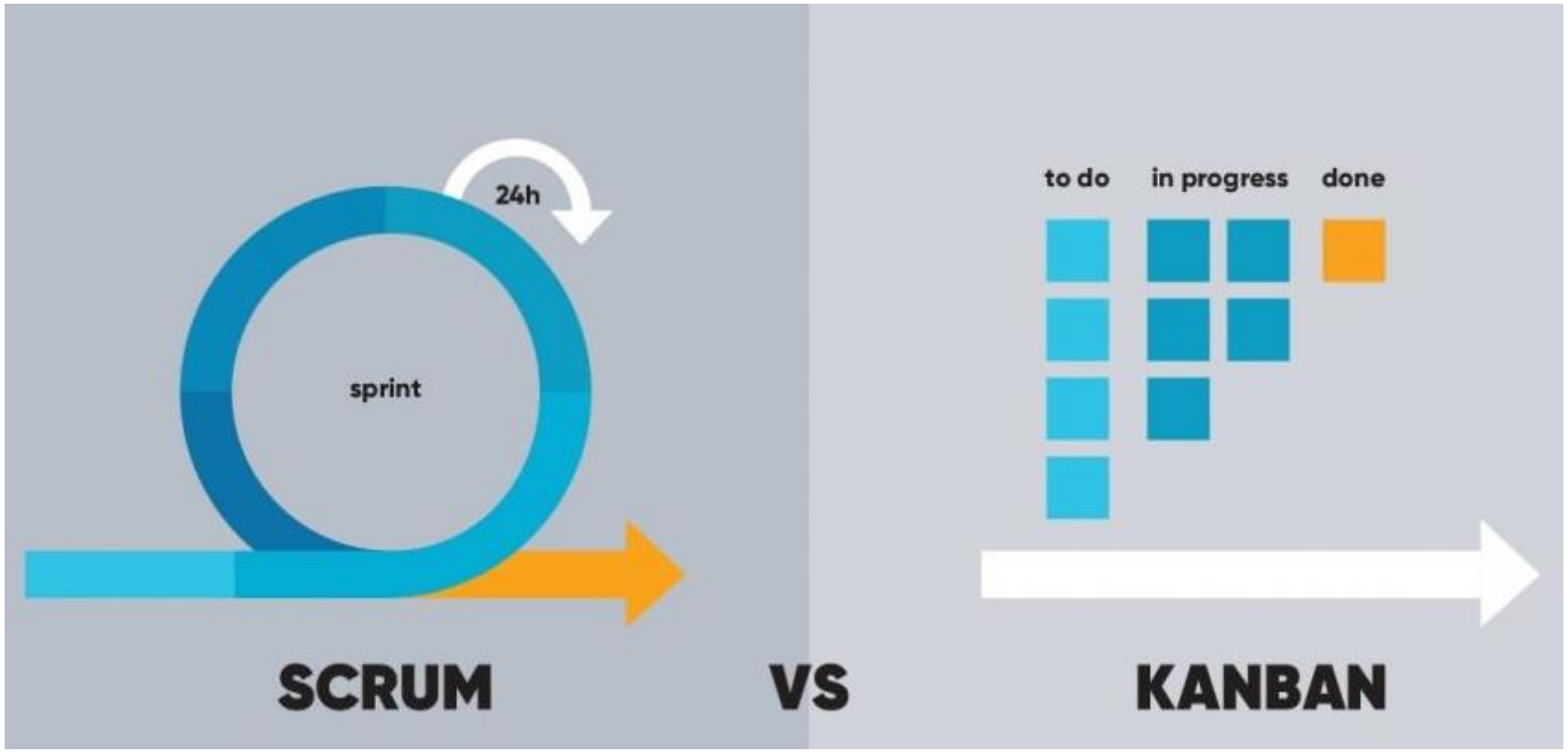
In fact, Kanban was developed long before Agile was even a thing. It was first developed in the 1940s by a Toyota executive named Taiichi Ohno, and is today recognized as an "agile" project approach because it prioritizes similar values like adaptability, efficiency, and improved collaboration.

The Kanban approach is a means of visualizing a project's status and progress via Kanban boards. (The Japanese word "kanban" means "billboard.") A Kanban board is divided into columns representing different stages of completion, like "not started," "in progress," and "complete." Tasks are sorted into columns and tagged with information, like priority level, person working on it, and project status.

Kanban Overview



SCRUM VS KANBAN – A FAIR COMPARISON



What are the differences between Scrum and Kanban?

- **Scheduling:** In the Scrum approach, managers set sprints by determining clear deadlines for each aspect of the project. Kanban, on the other hand, encourages continuity with the responsibility of setting deadlines lying with the team leader.

- **Duties:** In Scrum, every team member is delegated with a specific task. However, in Kanban, roles for team members are not strictly determined, resulting in greater cooperation.

- **Improvisations:** Scrum does not really allow improvisations and changes in the middle of a sprint, while Kanban encourages modifications and promotes continuous adjustments.

- **KPIs:** The velocity of a sprint helps in understanding productivity in Scrum. And since sprints are the smallest pieces of work, they get executed very quickly. Kanban, on the other hand, considers the time taken to complete a larger portion of the project.

- **Application:** While Scrum works best with projects that have fixed deliverables, Kanban is better suited for flexible teams that need frequent changes in-between.